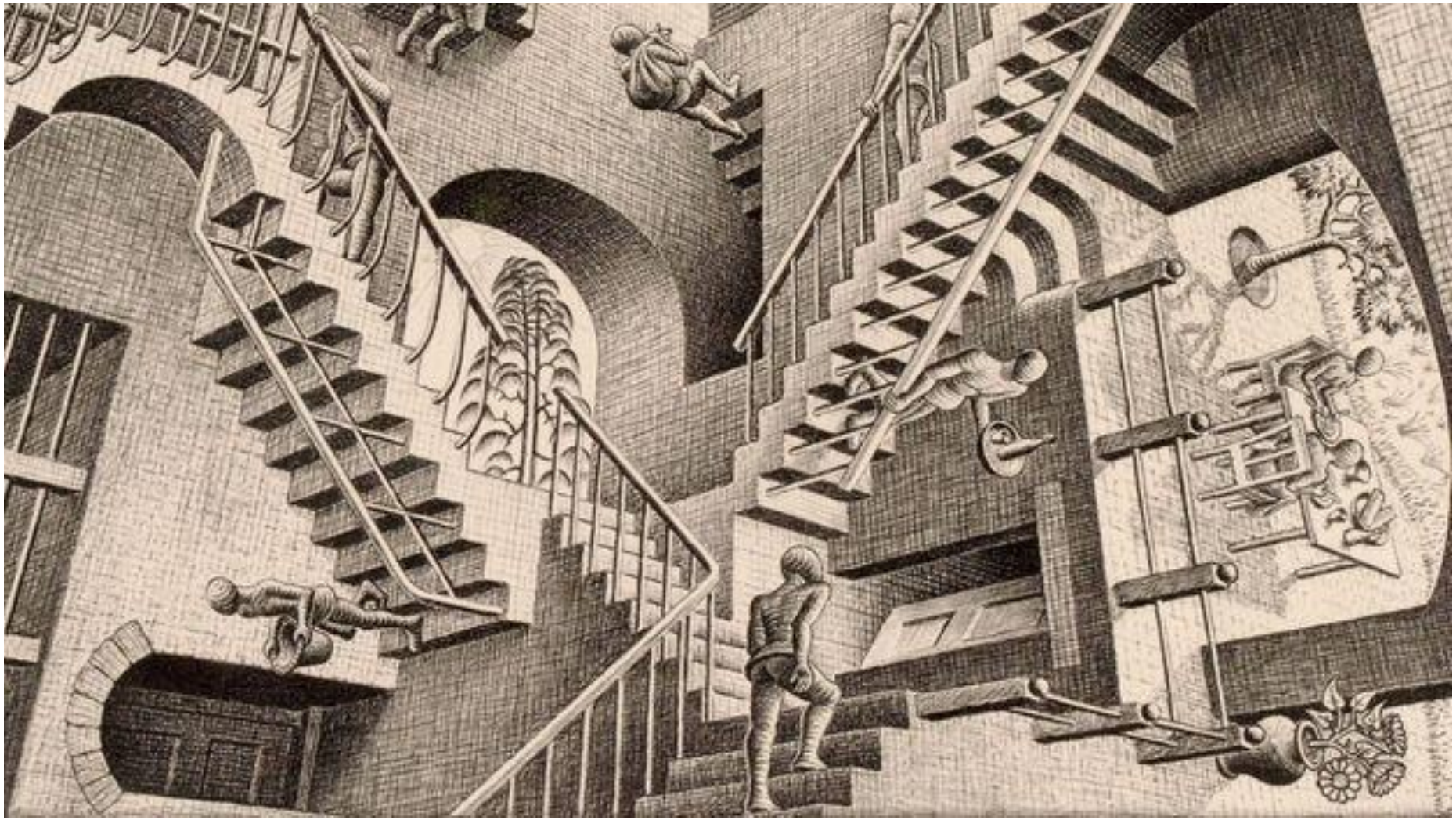
A Gentle Intro to Sim2Real

ESE 6510
Antonio Loquercio

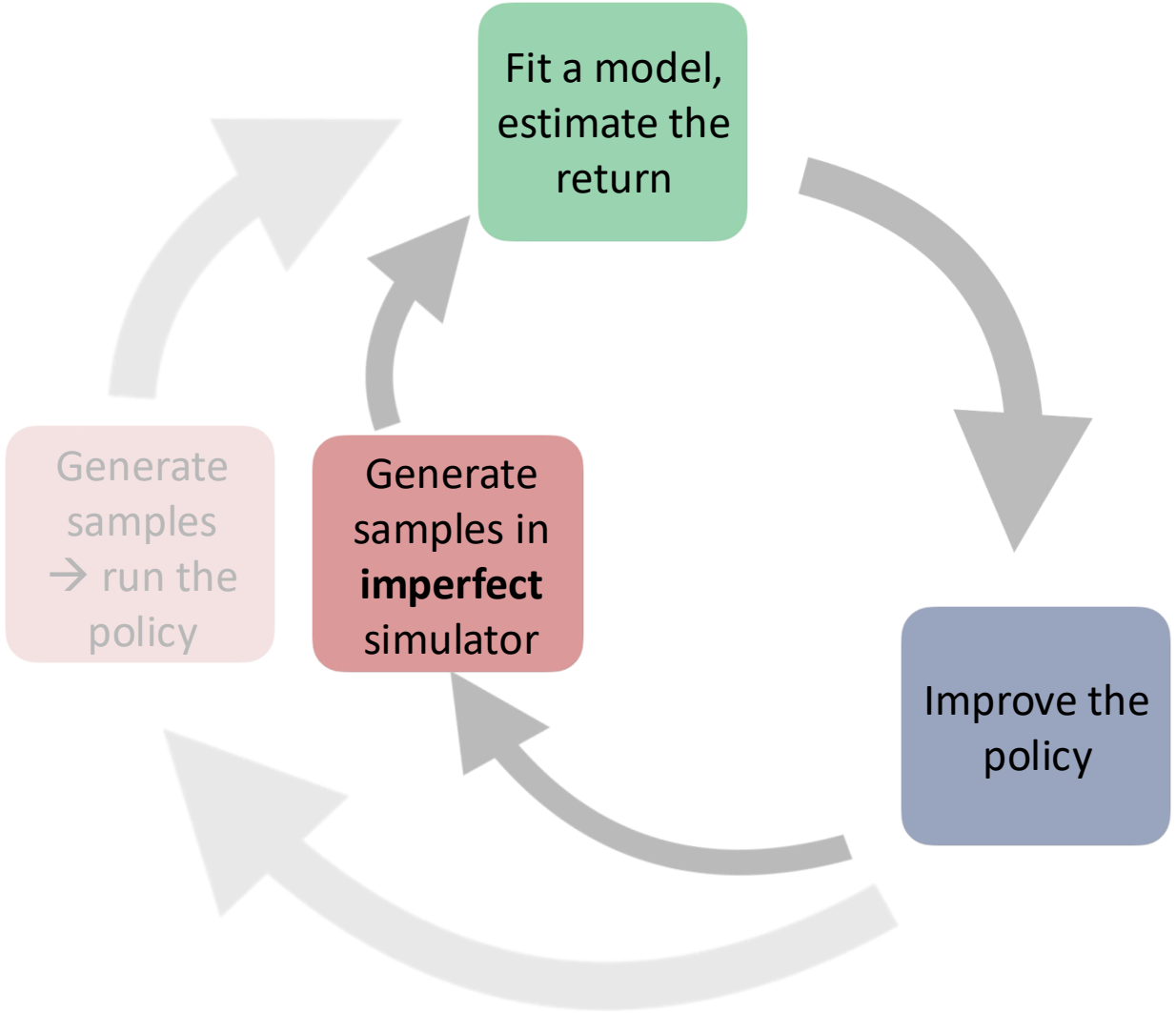


M.C. Escher,

***Relativity*, 1953**

What is Sim2Real?

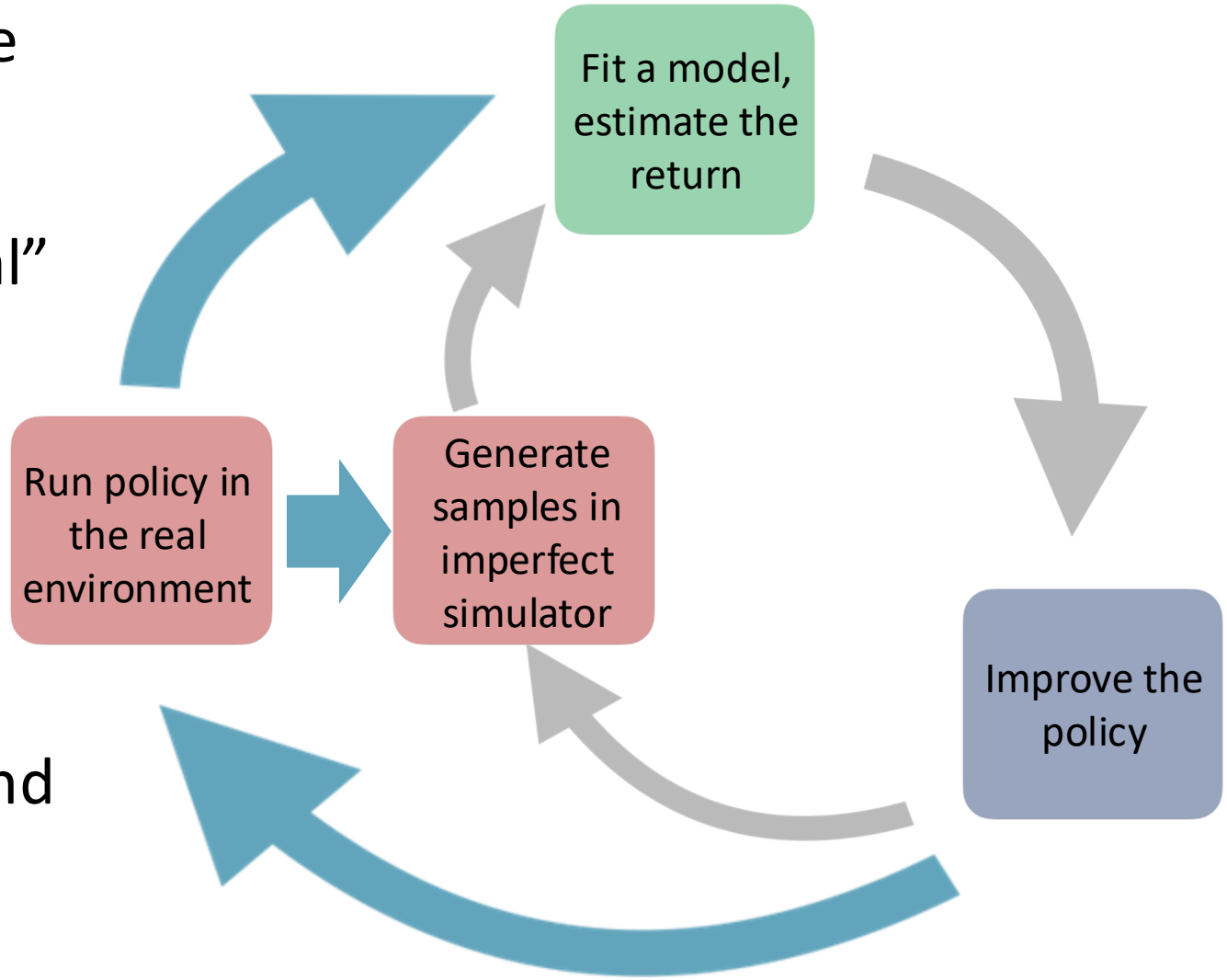
A family of methods that enable policies trained in an **imperfect world model** to successfully transfer and operate in the “real” environment.



What is Sim2Real?

A family of methods that enable policies trained in an **imperfect world model** to successfully transfer and operate in the “real” environment.

Evaluations of the policy in the real environment are used to **improve** both the **simulation** and the **policy learning** approach.



What Sim2Real is NOT

- Equivalent to model-based RL:
  - Model-based RL is a super-set of sim2real. The latter is specifically tailored to imperfect simulators.

- A way to train policies with PPO in physics-based simulators.
  - Why only PPO? Can't I train a policy with other methods (e.g., imitation)?
  - Why should the simulator be physics-based? Can be data-driven or hybrid.

- A trick to train policies in fast simulators
  - You could have advantages even if the model is slower than the real environment.

Why Sim2Real?

- Faster/Safer to run a policy
  - Especially important for partially trained policies
- Focus policy learning on the subset of the environment dynamics that matter the most for the task at hand.
- Fine-grained control of policy experience/curriculum.
- Explore counterfactuals (what if one of the legs breaks?)

The Spectrum of Simulators (or World Models)

Physics-based

Data-Driven

GENESIS

Too many to list

MuJoCo

ManiSkill

...

General / Ease of Customization / Faster

Visual Fidelity / Environment Diversity

The Simulation to Reality Gap(s)

- Dynamics gap:

$$s_{t+1} = f_{real}(s_t, a_t) \neq f_{sim}\,(s_t, a_t)$$

- Sensory gap:

$$\mathrm{o_t} = obs_{real}(s_t) \neq obs_{sim}(s_t)$$

- Semantic (or Environment) gap:
  - Things should have the right size relatively to each other and placed where it makes sense.
  - Other agents should behave in a way that is realistic.

The Semantic (or Environment) Gap

Categories of Sim2Real Algorithms

- Domain Randomization
  - Naïve
  - Adaptive
  - Physics-based
- Adaptive Control
  - Explicit System Identification
  - Implicit System Identification
- Abstractions-based
  - Visual Abstractions
  - Control Abstractions

Most papers use a mix of these techniques. But some applications require care.

Domain Randomization

- **Key Idea**: Randomize the parameters of the simulator that are important for my task and I am uncertain of.

Dynamics

Examples: Mass, Friction, Restitution Coefficient. Motor Efficiency

Sensory

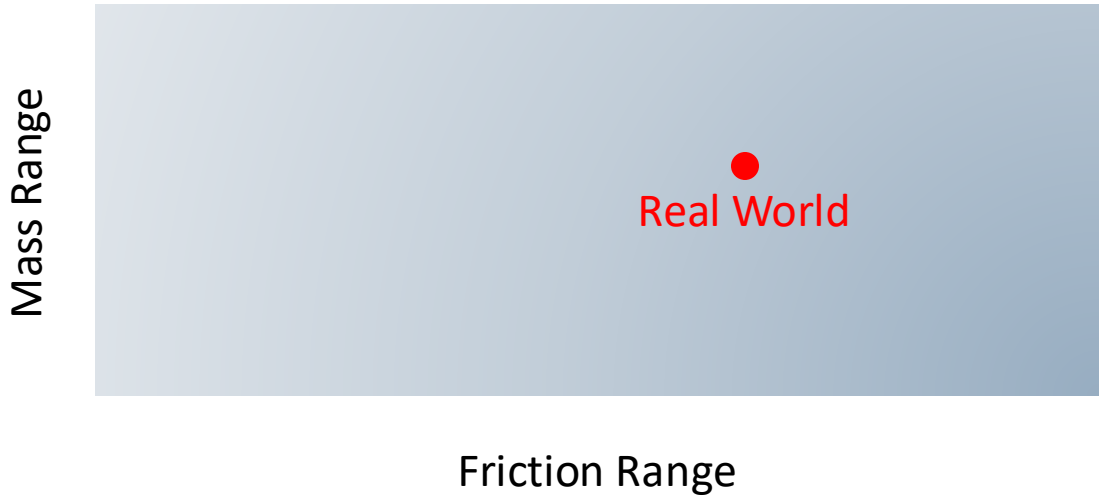
Examples: Latency, sensor noise, accuracy

Semantics

Examples: Background, Illumination, Object Types & Location

Domain Randomization

- **Popular Justification**: We randomize parameters with the hope that the real world is in the simplex of the parameters distribution ranges.



Why I think this is wrong (disclaimer: opinion not a fact)

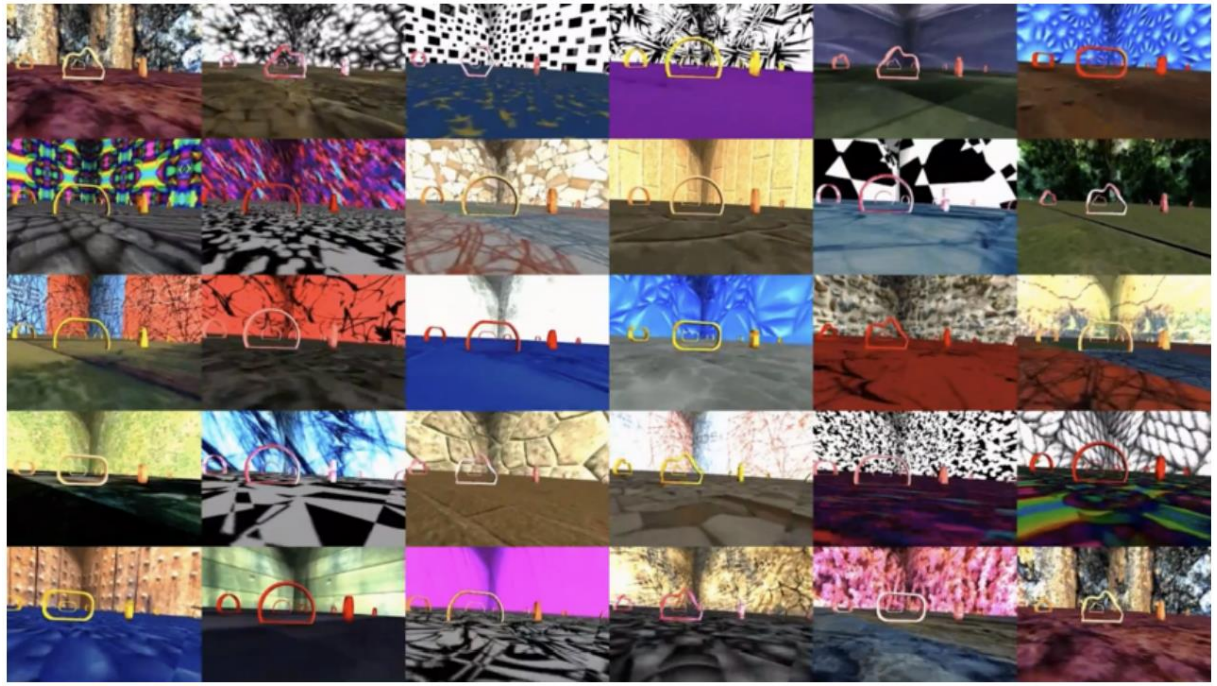
- Even if you randomize everything extensively, a simulator might still not capture the true dynamics due to its inner workings (numerical integration, collision handling, force computation).

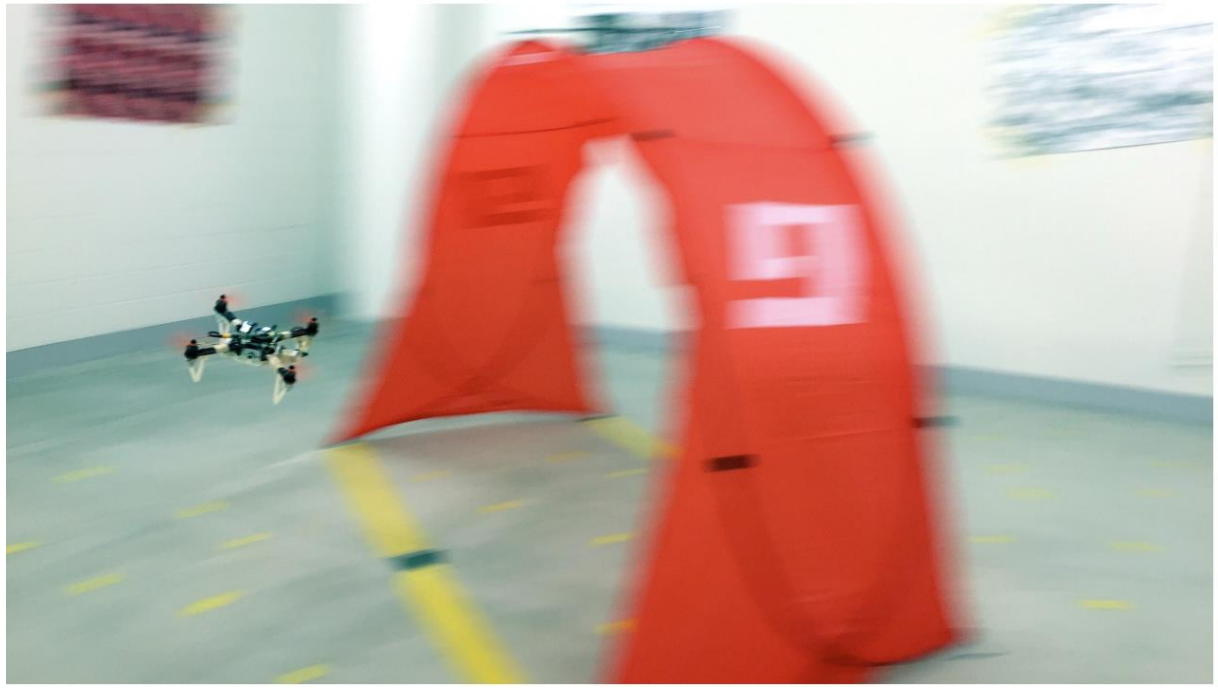
- The randomization ranges required might be so large that the probability of hitting a neighborhood of the real world is almost zero.

- It can be true at best for dynamics and sensory randomization, since they are physical parameters with a ground truth. What about the semantics randomization?

Only one of these must be true to disprove the argument!

Domain Randomization: An Example





Deep Drone Racing: from Simulation to Reality with Domain Randomization, T-RO 2019

Domain Randomization

- Objective: maximize the expected RL cost given the distribution of environment parameters.

$$\theta^* = argmax_\theta \, \mathbb{E}_{e \sim D}[\, C(\theta, e)]$$

Where:

- θ are the (trainable) parameters of the policy
- e are the environment parameters that control the dynamics (friction, mass, etc.), i.e., $s_{t+1} = f(s_t, a_t, e)$.
- D is the distribution of parameters we sample from.
- $C(\theta, e) = \mathbb{E}_{\tau(e,\theta)}[r(s_t, a_t)]$ is the RL cost.

Domain Randomization vs Robust Control

- Domain Randomization maximizes the objective in expectation:

$$\theta^* = argmax_\theta \, \mathbb{E}_{e\sim D}[\, C(\theta, e)]$$

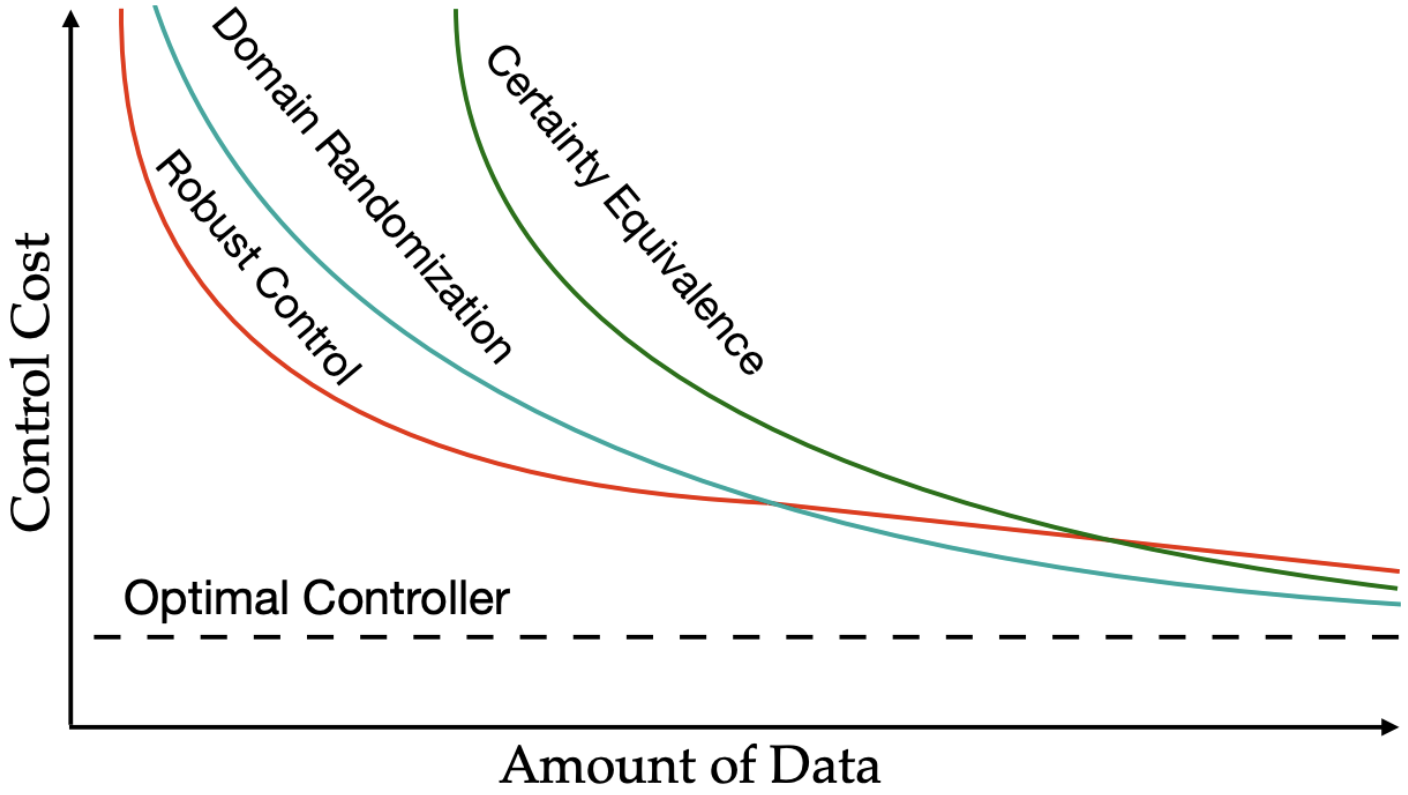
- Robust control maximizes the worst-case scenario of the objective.

$$\theta^* = argmax_\theta \, min_e [C(\theta, e)]$$

- Domain Randomization is less conservative than Robust Control. But in both cases, the larger the support of the parameter distribution, the harder the task.

Burn-in time vs Cost at Convergence

You can theoretically and empirically show that the relation below for a linear system and LQR controller (no RL).



Domain Randomization is Sample Efficient for Linear Quadratic Control, 2025

How to pick the parameters distribution?

- Naïve
- Performance-based (Adaptive)
- Physics-based

How to pick the parameters distribution?

- Naïve: Select the subspace of parameters you are the most uncertain about. Fix the ones you know.

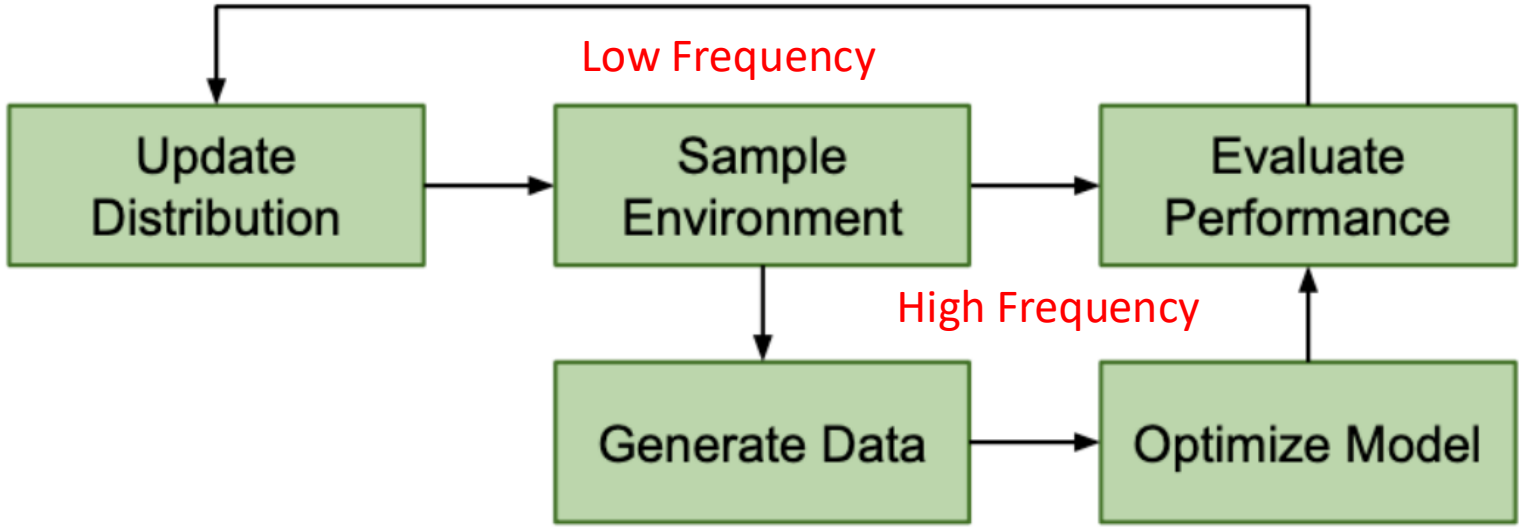
- If no prior information is available, use a uniform distribution over the range of parameters.

Limitations?

- Compute intensive/challenging to converge if the ranges are large.
- Some combinations of parameters potentially make no sense; why train on them?

How to pick the parameters distribution?

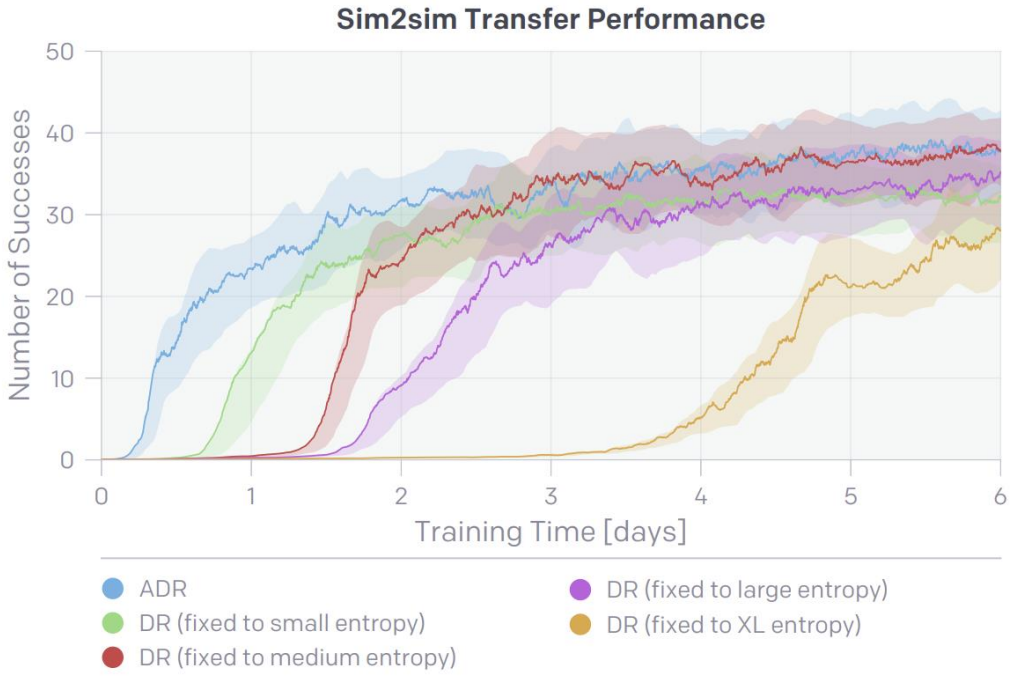
- Performance-based (adaptive): Gradually adapt the ranges to make the agent never too successful or unsuccessful: Curriculum-like.

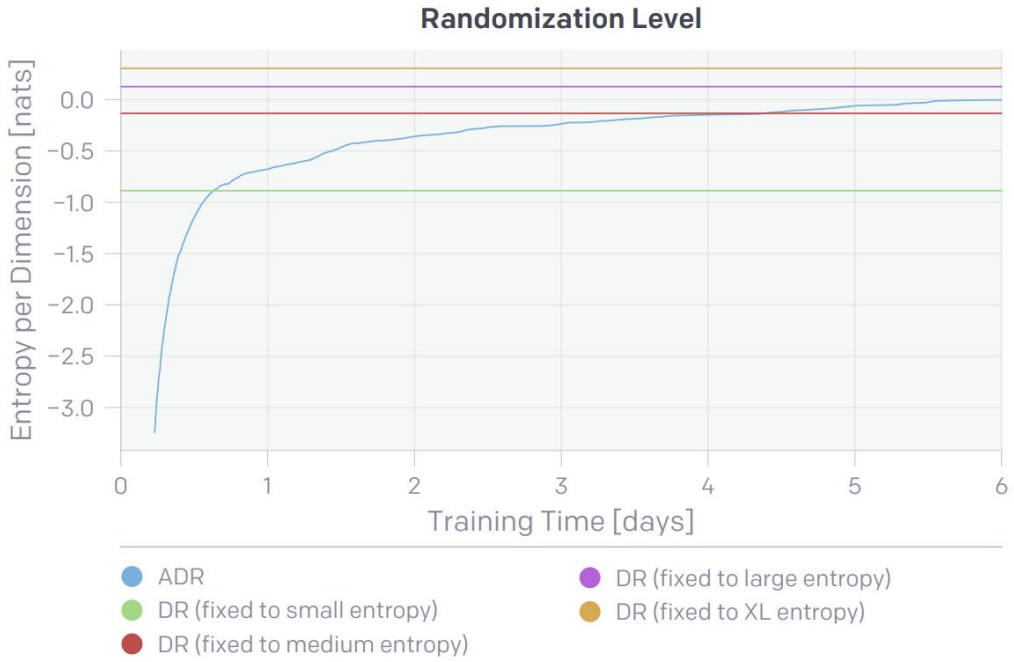


- Popularized by OpenAI's Rubik's Cube paper (https://openai.com/index/solving-rubiks-cube/)

How to pick the parameters distribution?

- Performance-based (adaptive): Faster to converge than traditional DR





Solving Rubik's Cube with a Robot Hand, 2019

How to pick the parameters distribution?

- Performance-based (adaptive): Can train on larger ranges and therefore transfer better (note: comparison not 100% fair).

Policy	Training Time	ADR Entropy	Successes (Sim)		Successes (Real)	
			Mean	Median	Mean	Median
Baseline (data from [77])	—	—	43.4 ± 0.6	**50**	18.8 ± 5.4	13.0
Baseline (re-run of [77])	—	—	33.8 ± 0.9	**50**	4.0 ± 1.7	2.0
Manual DR	13.78 days	-0.348^{*} npd	42.5 ± 0.7	**50**	2.7 ± 1.1	1.0
ADR (Small)	0.64 days	-0.881 npd	21.0 ± 0.8	15	1.4 ± 0.9	0.5
ADR (Medium)	4.37 days	-0.135 npd	34.4 ± 0.9	**50**	3.2 ± 1.2	2.0
ADR (Large)	13.76 days	0.126 npd	40.5 ± 0.7	**50**	13.3 ± 3.6	11.5
ADR (XL)	—	0.305 npd	45.0 ± 0.6	**50**	16.0 ± 4.0	12.5
ADR (XXL)	—	**0.393 npd**	$\mathbf{46.7 \pm 0.5}$	**50**	$\mathbf{32.0 \pm 6.4}$	**42.0**

Solving Rubik's Cube with a Robot Hand, 2019

How to pick the parameters distribution?

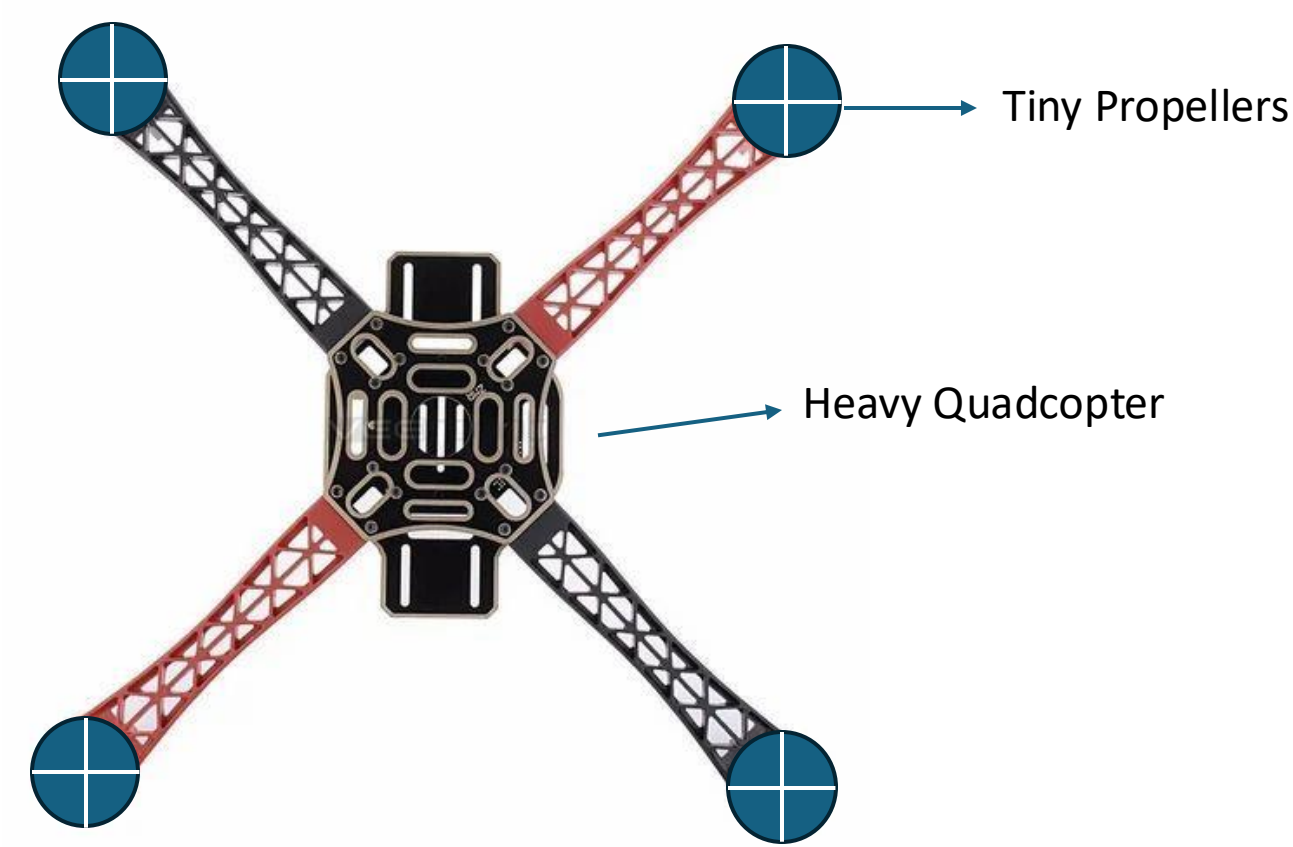
- Performance-based (adaptive): Not very much used in practice (but I have seen some good recent papers using it).

- Limitations?

- Many more hyperparameters beyond the ranges
  - Success/Failure thresholds, minimum waiting time, delta update, …
- To some extent, this happens already if you train with massively parallel environments:
  - At the beginning of training, the batch will be full of “easy” parameters but later it will balance.

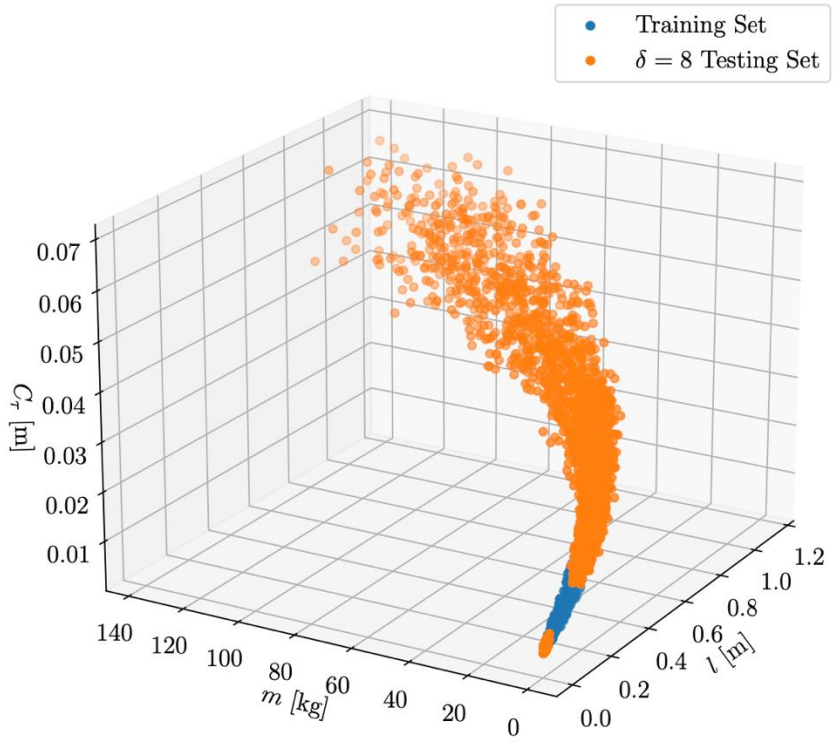
How to pick the parameters distribution?

- Physics-based: Use prior knowledge about the system to generate parameter combinations that are physically plausible.
- Why train on this?



How to pick the parameters distribution?

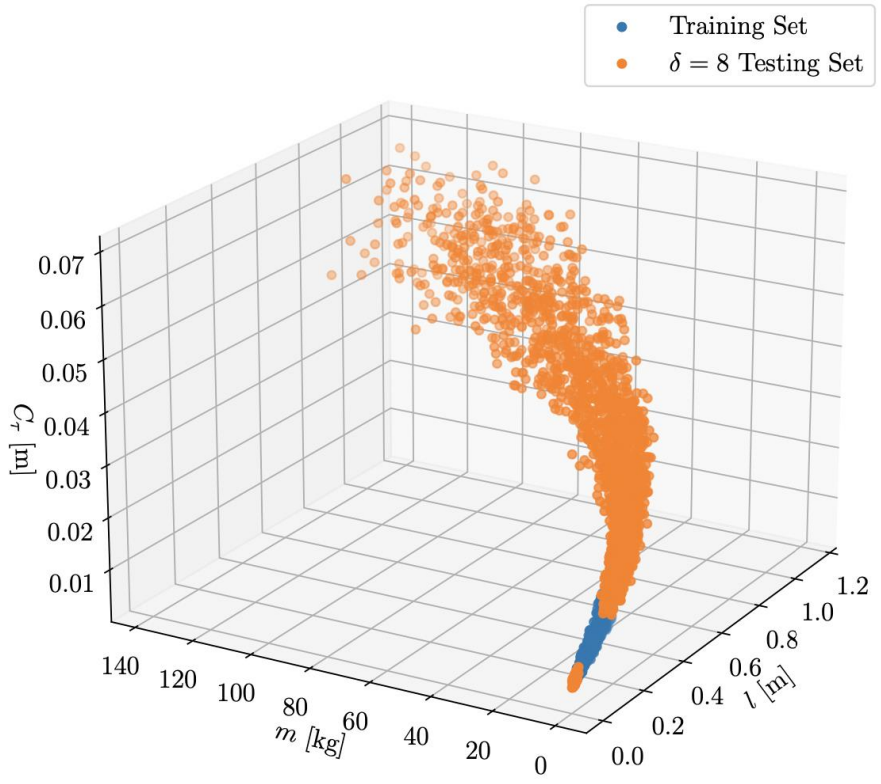
- Physics-based: Use prior knowledge about the system to generate parameter combinations that are physically plausible.

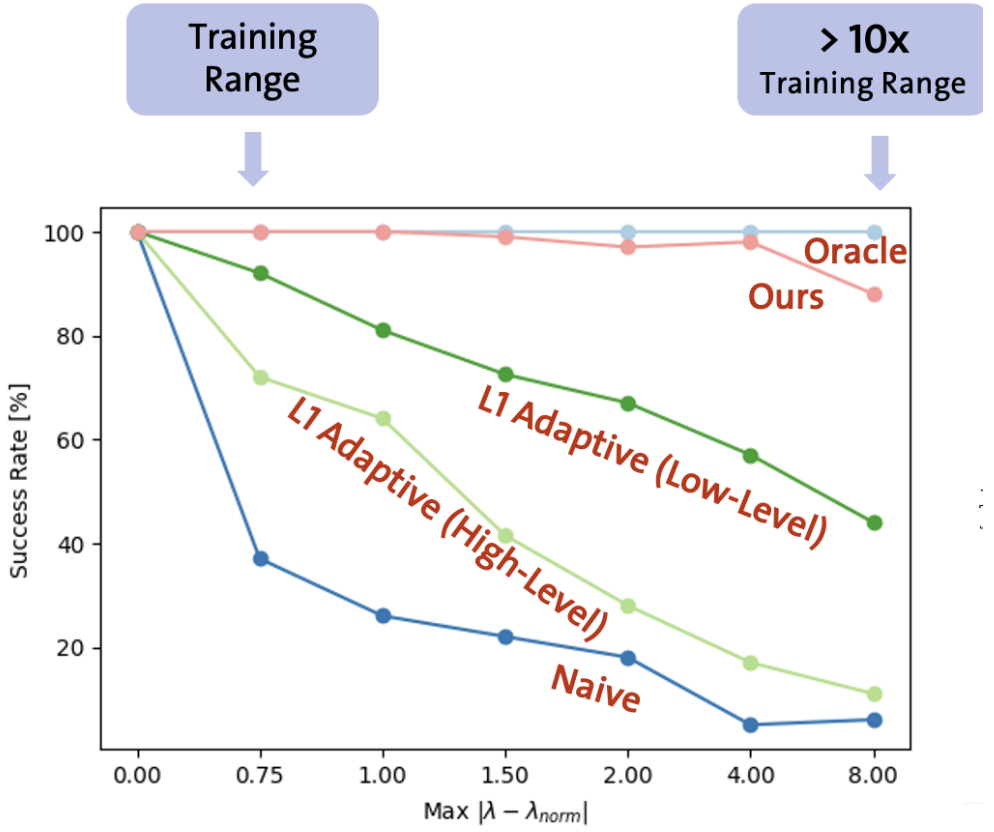


A Learning-based Quadcopter Controller with Extreme Adaptation, 2025

How to pick the parameters distribution?

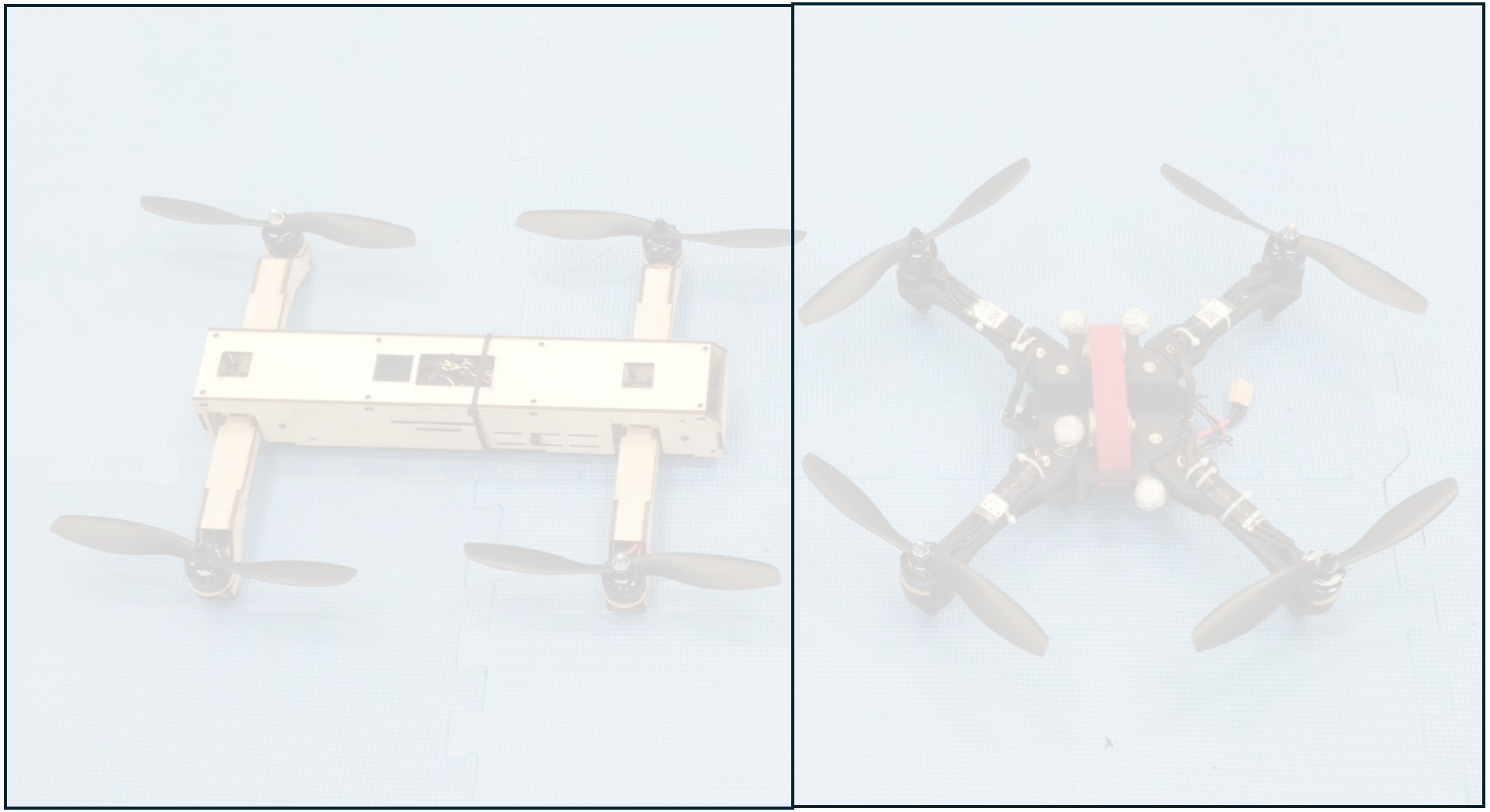
- **Interesting (but problem-specific) finding.** Physics-based randomization enables generalization beyond the training range.



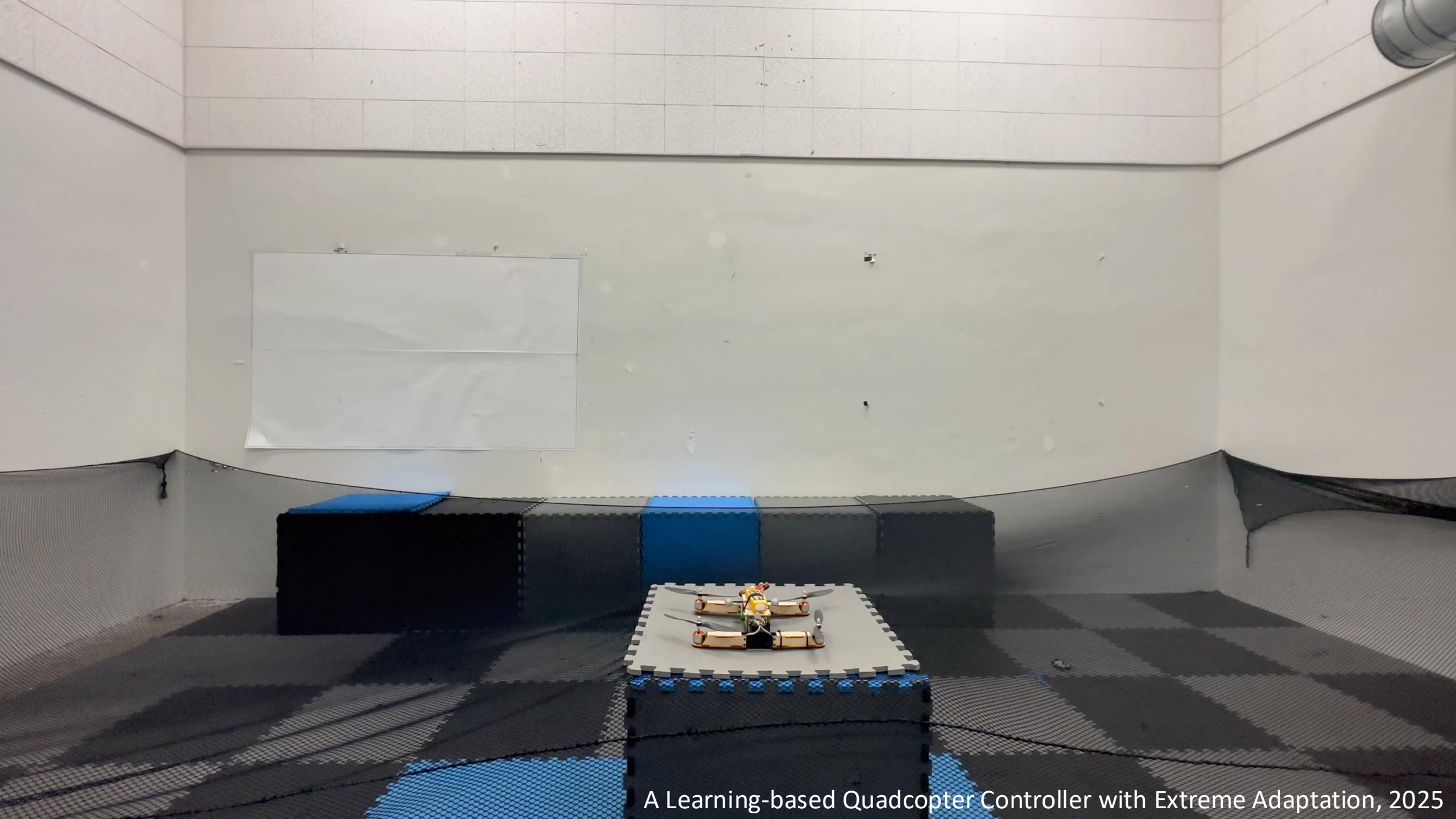


A Learning-based Quadcopter Controller with Extreme Adaptation, 2025

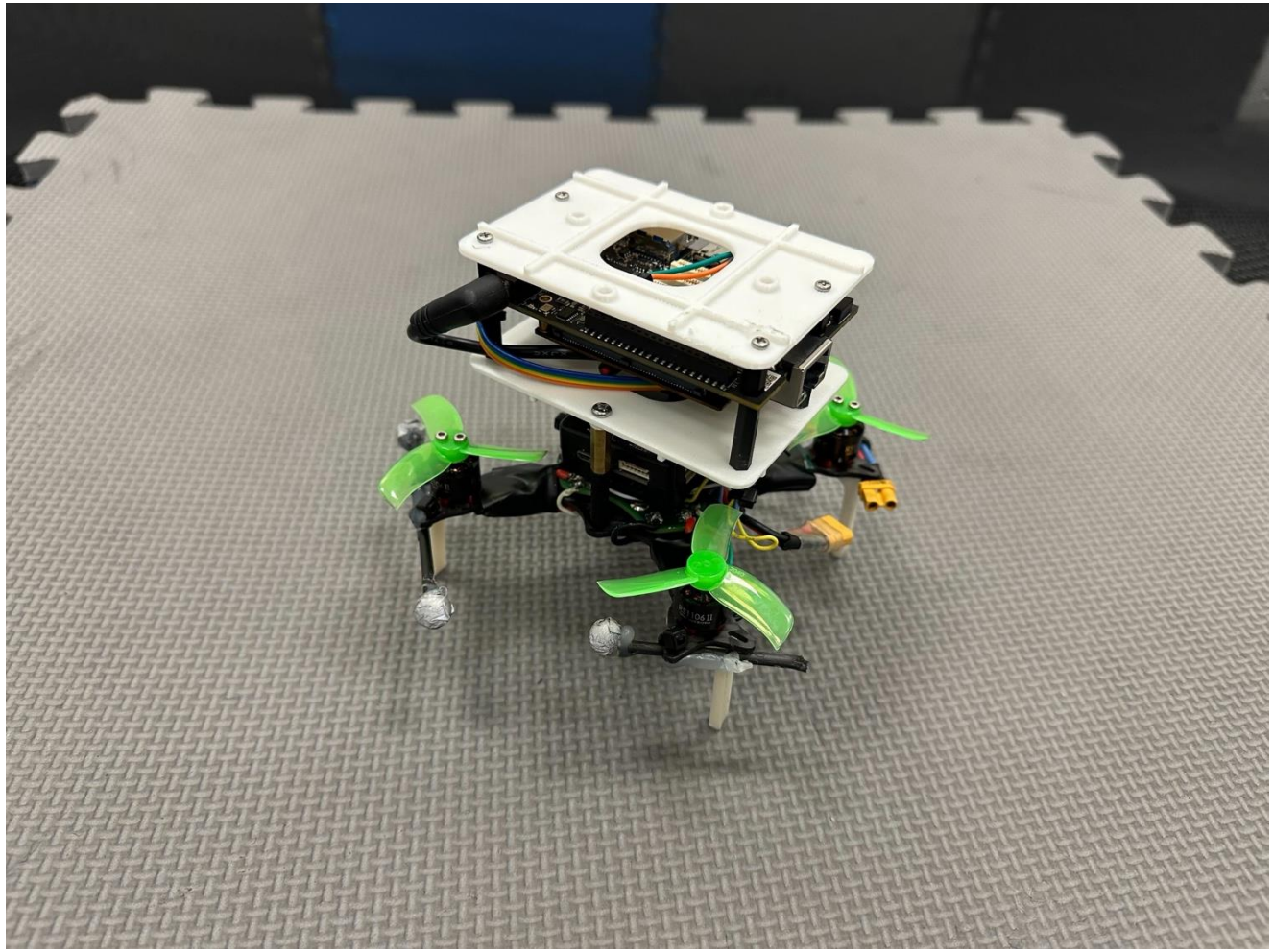
Flying Different Morphologies



QUaRTM: A Quadcopter with Unactuated Rotor Tilting Mechanism Capable of Faster, More Agile, and More Efficient Flight, Jerry Tang, Karan P. Jain, and Mark W. Mueller

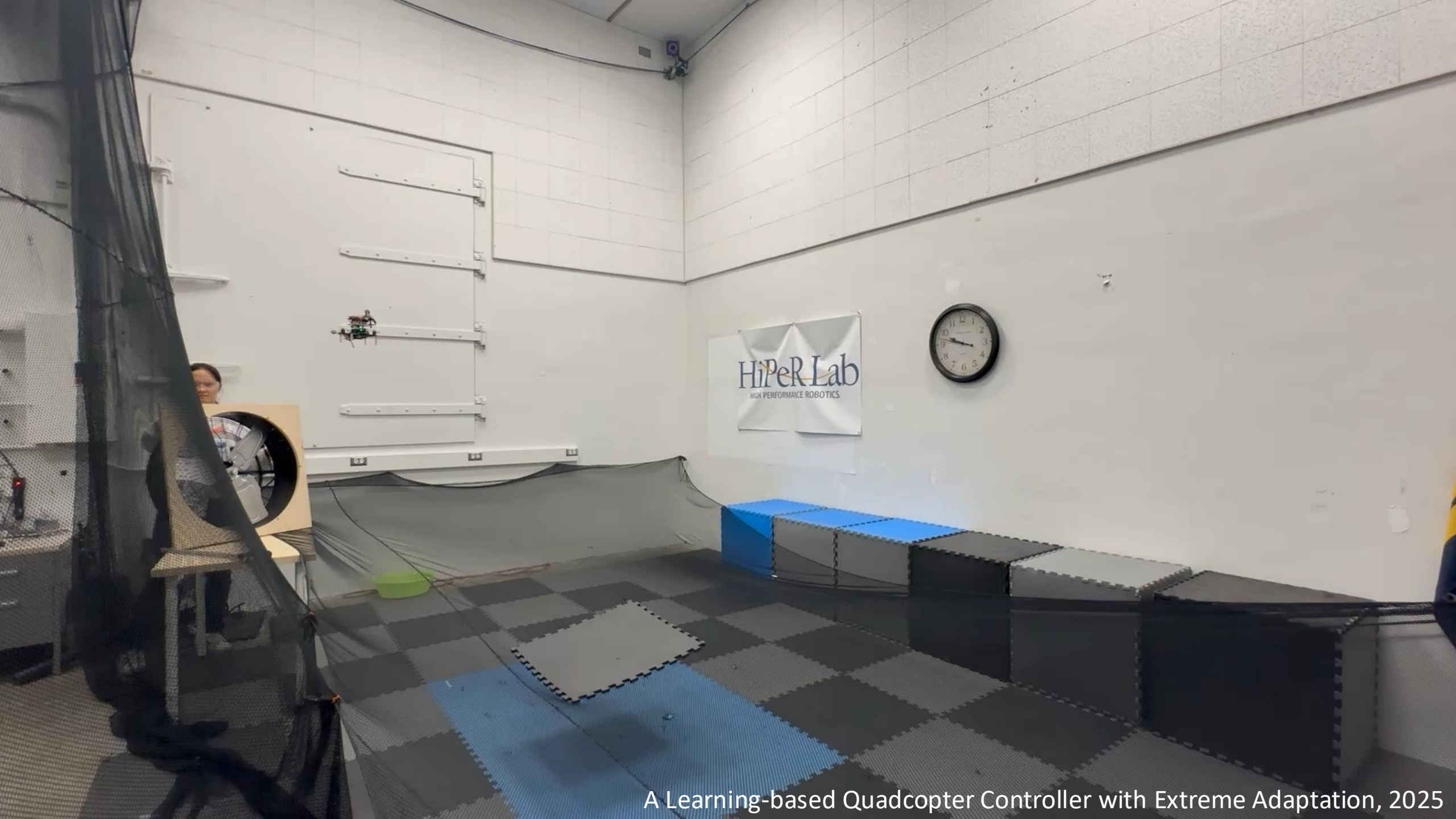

A Learning-based Quadcopter Controller with Extreme Adaptation, 2025

Flying Different Morphologies



- ~30% mass above the propellers
- 3X larger x-z inertia than a normal drone.

A Learning-based Quadcopter Controller with Extreme Adaptation, 2025



A Learning-based Quadcopter Controller with Extreme Adaptation, 2025

How to pick the parameters distribution?

- Physics-based: Use prior knowledge about the system to generate parameter combinations that are physically plausible.

- Limitations?

- Unclear how to design these relationships for complex systems (e.g., a dexterous hand).
- Unclear how to apply this for sensory and semantic randomization.

Why does domain randomization work? (Disclaimer: Opinion)

- Let's go back to the optimization objective of domain randomization

$$\theta^* = argmax_\theta \, \mathbb{E}_{e \sim D}[\, C(\theta, e)]$$

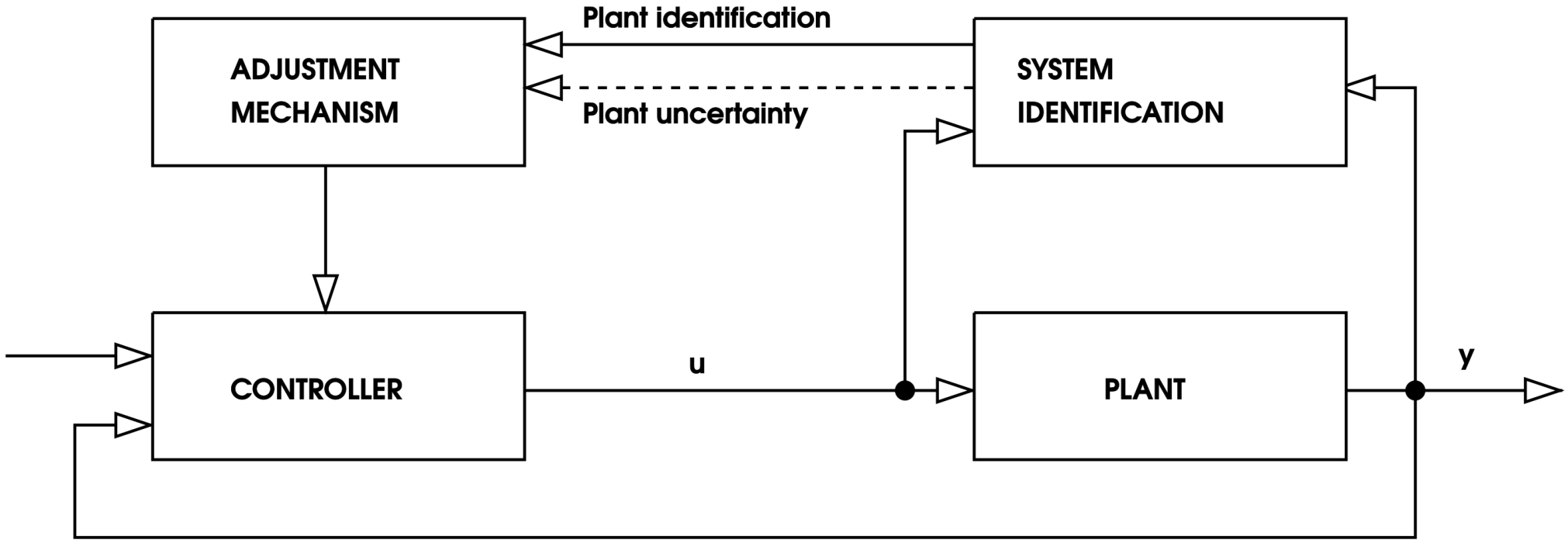
- A good solution to this problem is learning to quickly learn/adapt.
- For example, the agent can learn how to do quick estimation of parameters and learn the relation between parameters and actions (e.g., the larger the mass, the larger the force).
- This does not require that the real world is in the simplex of the parameter ranges.
- Can we design algorithms to bias the agent to learn this behavior?

Categories of Sim2Real Algorithms

- Domain Randomization
  - Naïve
  - Adaptive
  - Physics-based
- Adaptive Control
  - Explicit System Identification
  - Implicit System Identification
- Abstractions-based
  - Visual Abstractions
  - Control Abstractions

Adaptive Control

- **Key Idea**: Identify the parameters of the system from (possibly limited) on-policy experience. Condition or finetune the policy on the estimated parameters.



MODEL IDENTIFICATION ADAPTIVE CONTROL (MIAC)

Source: Wikipedia

Adaptive Control

Two types of adaptive sim2real:

- **Explicit parameter estimation (Real-to-Sim)**: find the actual parameters.

- **Implicit parameter estimation:** find a compressed representation of the parameters.

Most methods do a hybrid of these two (identify what is fixed, adapt to things that change).

Adaptive Control: Explicit Parameter Estimation

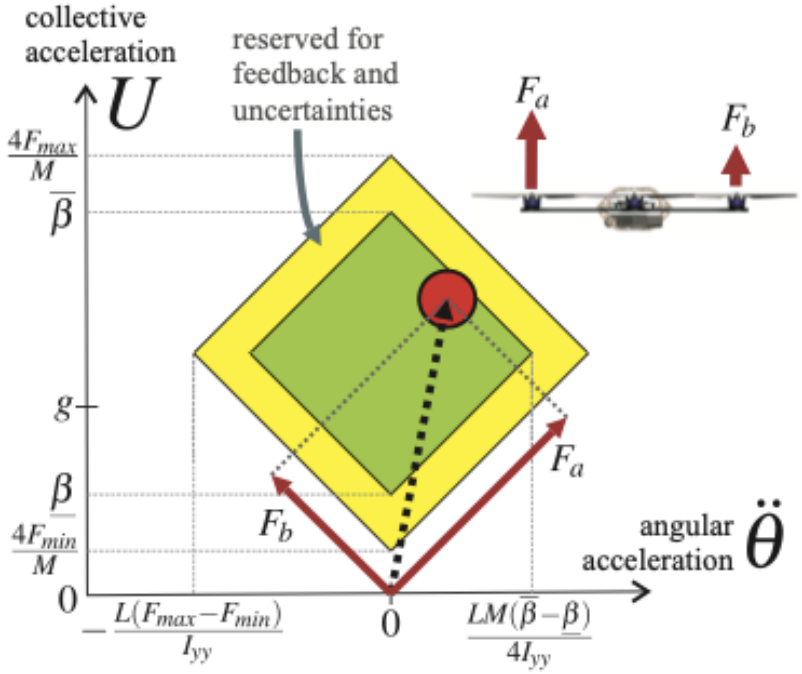
A Simple Learning Strategy for High-Speed Quadrocopter Multi-Flips

Sergei Lupashin, Angela Schöllig, Michael Sherback, Raffaello D'Andrea

ICRA, 2010

Adaptive Control: Explicit Parameter Estimation

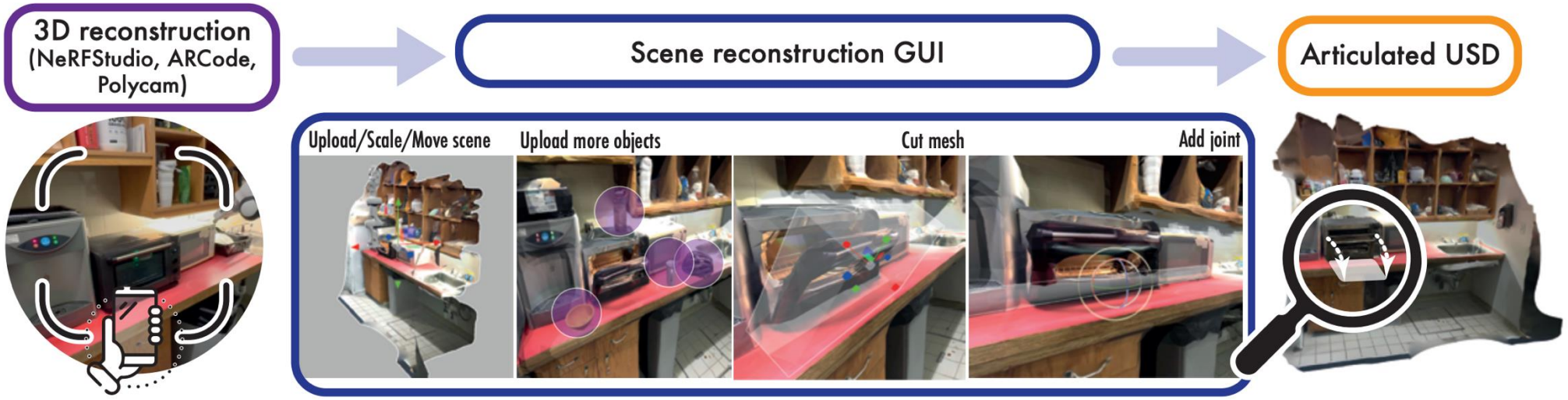
- Estimation is never perfect: uncertainty in some parameters always remains.
- To account for this, explicit identification is generally coupled with domain randomization.



A Simple Learning Strategy for High-Speed Quadcopter Multi-Flips, 2010

Adaptive Control: Explicit Parameter Estimation

Modern take to the problem: estimate not only physical parameters, but the whole scene.



Reconciling Reality through Simulation: A Real-to-Sim-to-Real Approach for Robust Manipulation, 2024

Adaptive Control: Explicit Parameter Estimation

Modern take on the problem: estimate not only physical parameters, but the whole scene.

Limitations of scene-level real-to-sim?

- The more complex the scene, the slower the simulation.
- Articulation of objects is non-trivial when done for more than a handful of objects (particularly hard for deformable objects).
- Reconstruction artifacts might impact the behavior of the policy. Policies are generally finetuned, not trained from scratch.

However, we don’t (yet) have better methods to cope with the semantic sim2real gap!

Adaptive Control

- **Explicit parameter estimation (Real-to-Sim)**: find the actual parameters.

- Limitations?

- Some parameters might not be identifiable given some limited on-policy experience.
- Multiple parameters might explain the same behavior. Which one to pick?
- We might not really need the parameters by themselves, but by their relation.